City, University of London
Artificial Intelligence MSc
INM707 Deep Reinforcement Learning

1 Defining the Environment and Presenting the Problem

We took inspiration from Ocado Technology, a leading company in the robotics and AI market, to design our environment and problem. Ocado Smart Platform is a platform technology combining AI and robotics, among other technologies, to provide a Multi-Agent robotics infrastructure to completely automate the order picking task associated with online orders. OcadoGroup (2022) At the heart of their platform lies their rectangle grid split into cells and a team of robots, able to move left-right and forwards-backwards on this grid. Each cell has a container below it holding a specific item available for the robot to pick up and place in its cage. In the state of the art platform from Ocado, all robots are managed by a central algorithm, ensuring they don't crash into each other and that tasks are split in the most efficient way possible.

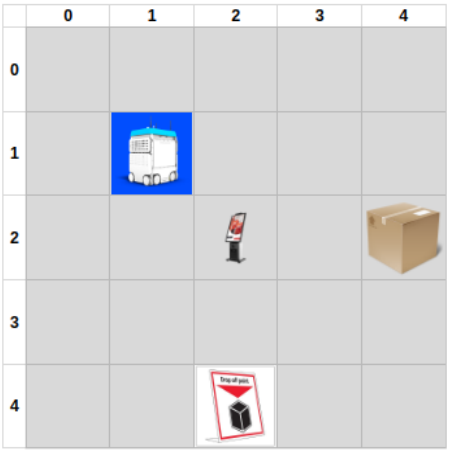


Figure 1: Birds eye view of a sample warehouse layout used in our investigations

In our investigation for our coursework, we used a simplified version of the Ocado Smart Platform. The main differences were that we had a single robot instead of many and our grid size was much smaller at 5×5.

The problem was to pick the ordered item without bumping into the walls or the warehouse kiosk and then drop in off at the drop off point in the shortest number of steps possible.

2 State Transitions and Rewards

As shown in figure 1, our 5×5 grid has a kiosk in the centre at (2, 2), the surrounding walls can not be passed and the item has to be dropped off at the lower boundary of the warehouse for shipping. As long as the robot doesn't bump into the boundary walls or the kiosk, it can move up, down, left and right. In addition to these moves it can also always attempt to pickup an item or drop it off (regardless of whether it is the correct location or whether a parcel is available in its cage to drop off.)

This gives our robot six possible actions and $5 \times 5 = 25$ cells it can attempt to visit minus 1 cell which houses the kiosk which it can not visit.

Each time our agent tried to make a move, its action was first checked by the algorithm to ensure that is was a legal move, ie., not trying to cross a boundary wall of bump into the kiosk, and them its current position was updated. If the selected actions were to pick up or drop off a parcel, the algorithm

updated the Boolean variable cage_full accordingly and if the parcel was dropped in a cell other than the drop off cell then the position of the parcel was also updated to keep track of where the parcel is and whether it has been picked up again or not.

Greedy Policies are dangerous! This policies can lead our agent to select same action again and again. In stochastic environments this action selection behaviour delays the learning. For our agent to learn to carry out its task efficiently, we tested the three sets of rewards and punishments given in Table 1.

Table 1: Reward Structure

Action	Rewards A	Rewards B	Rewards C
Bumping into a boundary wall	-10	-1	0
Bumping into the kiosk	-20	-1	0
Taking a legal step in any direction	-1	0	0
Attempting to pick up a parcel where there is no parcel	-10	-1	0
Dropping off a parcel anywhere other than the drop off cell	-10	-1	0
Successfully picking up the parcel	20	1	0
Successfully dropping off the parcel in the drop off cell	50	1	1

The idea behind the three sets of rewards, A, B and C, was that A resembled a human understanding of what a worker should do in a warehouse. Set B was a simplistic good bad and neutral, giving simple signals to our agent. Lastly set C was an ultra simple attempt to let the agent to figure out how to achieve its goal without any intermediary rewards or uninformative reward structures. At the end we observed that the punishing a wrong action led us to a fast convergence since our agent tries to maximise the expected total return and when the reward signal is largely uninformative, the task suffers the problem of sparse rewards. The reward signal is the primary basis for altering the policy; if an action selected by the policy is followed by low reward, then the policy may be changed to select some other action in that situation in the future Sutton and Barto.

3 Q-Learning Parameters and Various Policies

3.1 Alpha

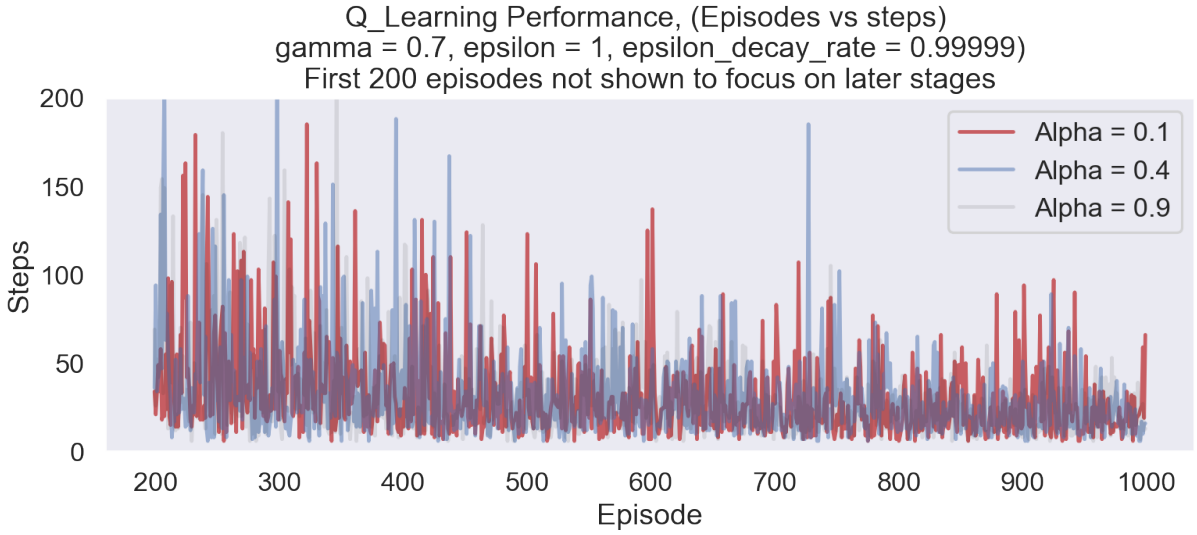


Figure 2: Looking for the best Alpha Value

In order to tune the Alpha hyper-parameter, the proportion of new Q values we update our Q matrix with at each update cycle, we tested various values of Alpha between zero and one. Figure 2

shows a low, medium and high Alpha's performance and upon closer inspection it is understood that larger values of Alpha result in consistently lower number of steps needed by the agent to complete its task. A likely explanation for the algorithm favouring quick and large updates to its Q matrix is that the agent doesn't have a large state space to explore so taking very small proportions of the new Q values at each update would be a wasteful way to learn in a short task in a small environment.

3.2 Gamma

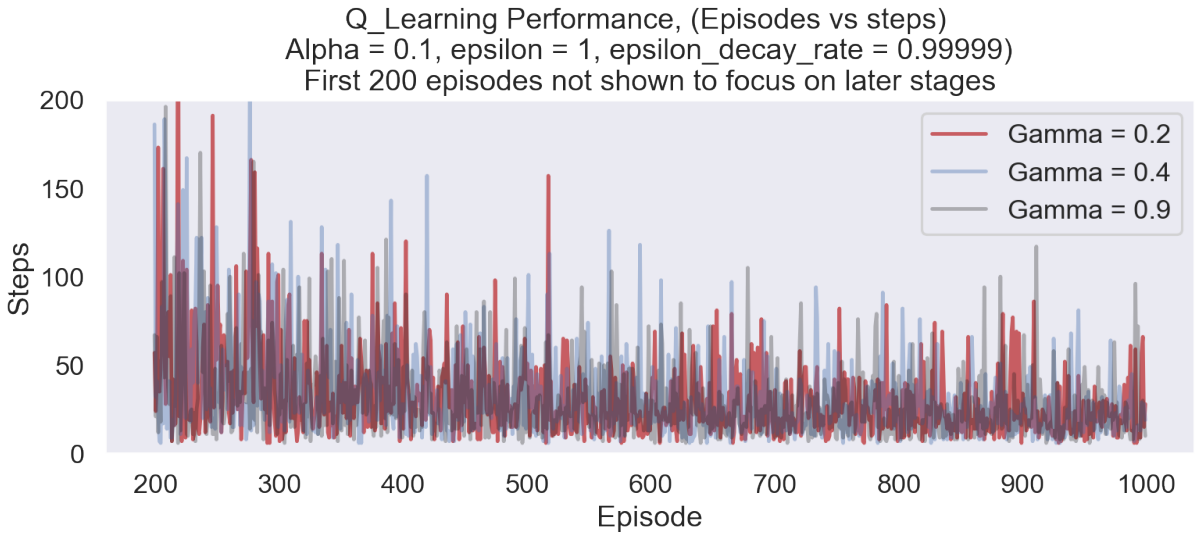


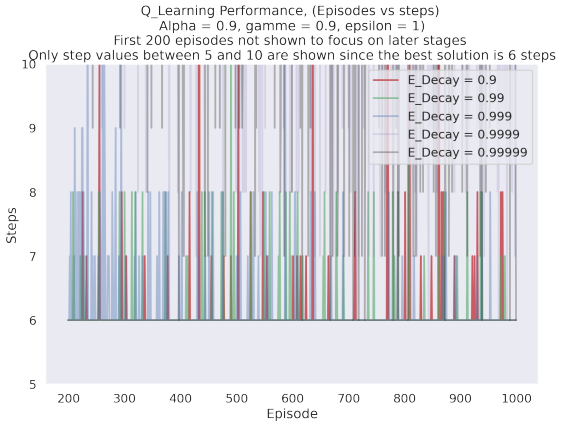
Figure 3: Looking for the best Gamma Value

Similarly whilst tuning Gamma, the discount factor for future rewards, in figure 3 it is seen that larger values of gamma result in quicker and more stable convergence at low step solutions to the task.

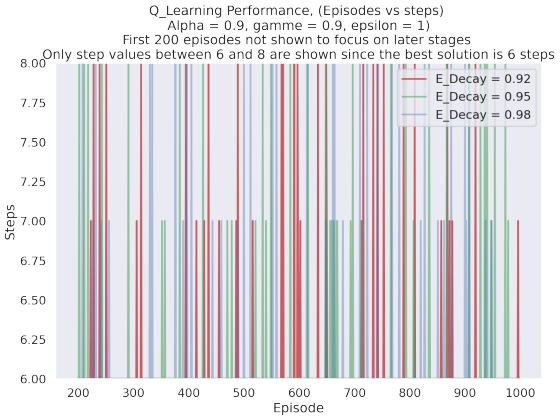
Both these hyper-parameters performed well whilst they were large (close to 1), a likely reason to this is the short nature of our task. For example, in the warehouse environment set up for these investigations, the least number of steps possible to complete the task is 6. This means that the agent making moves based on long term goals is futile because it can complete its task in just a few more steps.

3.3 Epsilon Decay Rate

Figure 4: Coarse and Fine graphs of various Epsilon decay rates



(a) Epsilon decay - steps 5 to 10 - Coarse Epsilon Decay rates



(b) Epsilon decay - steps 6 to 8 - Fine Epsilon Decay rates

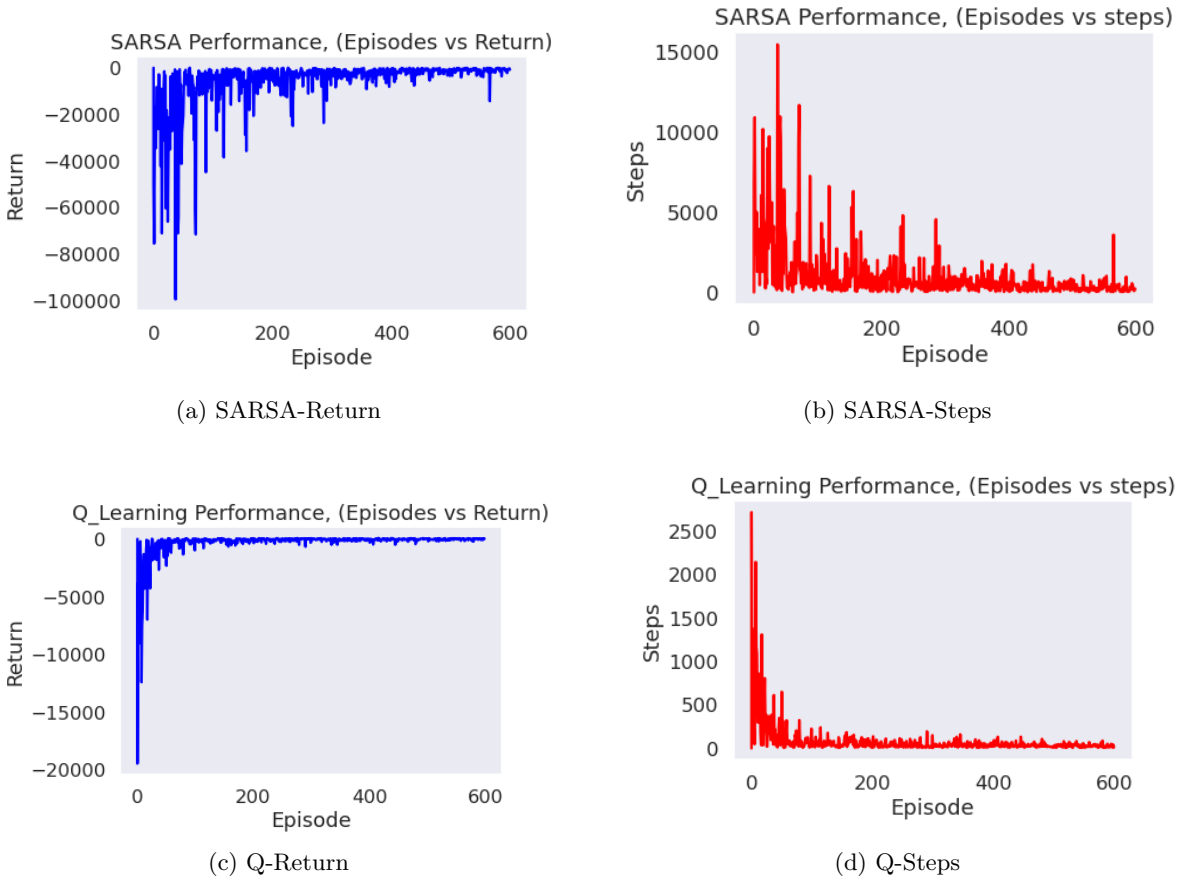
As seen in Figure 4(a), 5 different values were tested to tune the Epsilon Decay Rate hyper-parameter and 4(b) takes a closer look at the most successful range of values. Based on these results, the rest of the investigation used an Epsilon Decay Rate of 0.95.

3.4 Policies

We have tested two main policies which are Epsilon Greedy Policy and Softmax Policy. For the Epsilon Greedy Policy we further investigated by decaying the Epsilon value by 5 separate decay rates, details are given in 4. All of the Temporal Difference algorithms are guaranteed to converge to the optimal action -value function, as long as the step size parameter alpha is sufficiently small and the GLIE (greedy in the limit with infinite exploration) conditions are met. In practice it is common to completely ignore the GLIE conditions and we can still recover an optimal policy (Even-Dar et al., 2001). To show that we also tested not decaying it at all, which equates to the agent continuously exploring the environment and never exploiting it. In other words it is a completely random search and as expected it returned awful results not worth graphing. The random approach regularly required several thousands of steps to complete the task and always ended up with returns in the negative thousands. The other extreme would be a constantly greedy agent, but since such an agent never explores its environment it gives similarly poor results to its random counterpart. Whilst we coded a random_search function in our code, to see the results of a constantly greedy agent the epsilon decay rate could be set to 0 which would stop any exploration straight from the start of the first episode.

4 Q-learning algorithm performance

Figure 5: Initial performances of Q-Learning and SARSA Algorithms



(a) SARSA-Return

(b) SARSA-Steps

(c) Q-Return

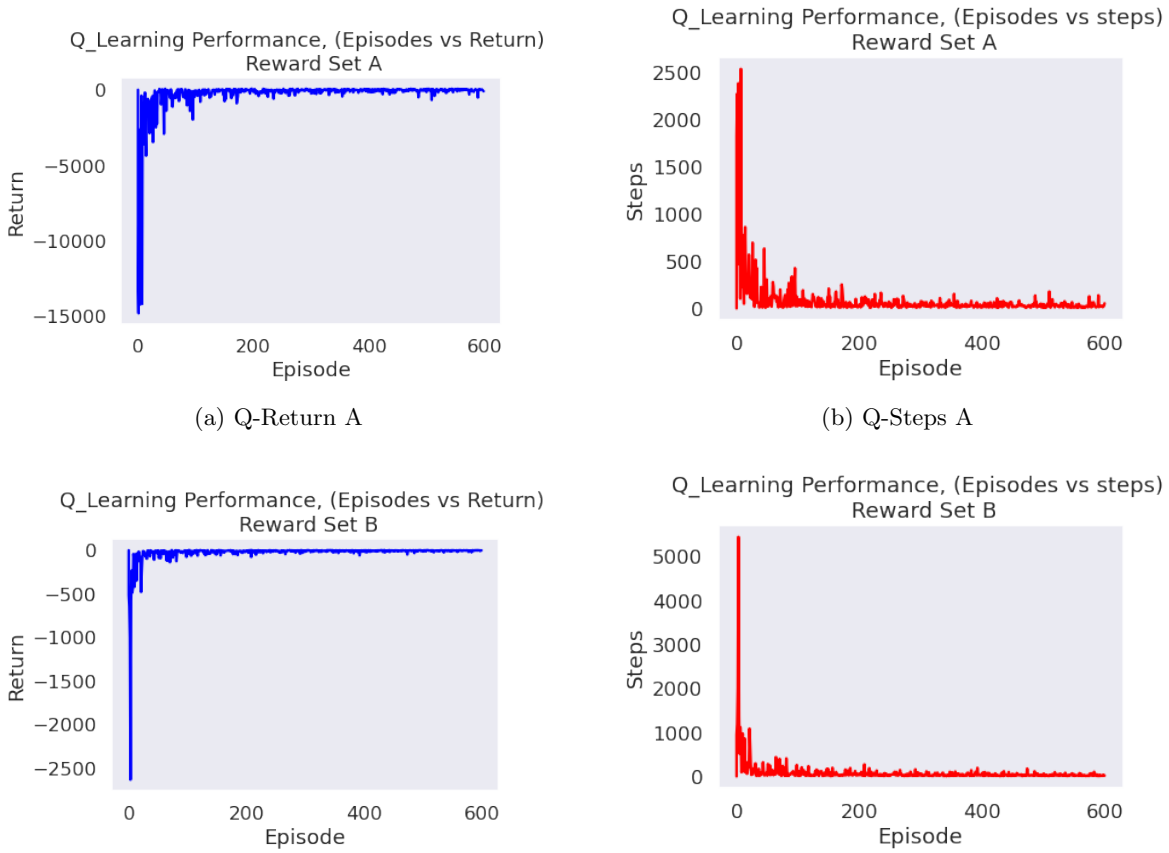
(d) Q-Steps

A quick inspection of figure 6 reveals that without extensive fine tuning the SARSA algorithm performed poorer than the Q-Learning algorithm. Whilst the softmax function used in the Q matrix

update for SARSA is used to get rid of the weakness of the excess randomness of the Q Learning algorithm, as stated in Sutton & Barto (2018) " Whether softmax action selection or ε-greedy action selection is better is unclear and may depend on the task ... "

5 Outcome of various reward structures

Figure 6: Performances of Q-Learning algorithm under Reward Structure A and B



(a) Q-Return A

(b) Q-Steps A

(c) Q-Return B

(d) Q-Steps B

To summarise the various hyperparameters used, we can see from Figure 2 that the larger value, 0.9, of alpha resulted in faster and more consistent convergence. Similarly figure 3 shows that the value 0.9 for gamma provided the best convergence in terms of steps needed to solve the task. Epsilon Decay rate was investigated in two steps, as shown in figure 4, and first we tried 5 coarse values ranging from $0.9 \rightarrow 0.99999$ and then based on the outcome of that we focused on the $0.9 \rightarrow 0.99$ range due to the better convergence between those values. Figure 4(b) shows that an epsilon decay rate of 0.95, green line, provided the most episodes with exactly 6 steps which is the best possible solution. Lastly inspecting figure 6, it becomes clear that the reward structure B provides much faster convergence to the optimal solution that reward structure A. Note that the results for reward structure C has been omitted because it didn't have any good results and took many orders of magnitude more to run without a maximum number of steps being allowed per episode.

In summary the optimal hyperparameters used are given below.

- Alpha = 0.9
- Gamma = 0.9
- Epsilon Decay Rate = 0.95
- Reward Structure = B

Both the high Alpha and Gamma values resulting in fast convergence as opposed to smaller values suggest that Updating the Q-Matrix with a large portion of new found knowledge is beneficial. This

is most likely dues to the small state space of our warehouse, $5x5$, and the low number of steps needed to solve the task, which is just 6 steps.

6 Analysis

Random search had very variable results but generally completed the task in between 1 and 10 thousand steps, since it explores all the time and has no sense of experience to maximise the total return, it gained approximately negative 7,000 rewards per 1,000 steps.

SARSA is an on-policy algorithm, meaning that the Q function/policy we are learning is the Q function/policy we are using in the environment. Our SARSA solution converged an optimal policy with a lack of experience. Because of its nature of update function it performed a bit better but didn't regularly achieve positive rewards

Q-Learning on the other hand is an off-policy algorithm, meaning that the policy we are learning are dictated by an epsilon greedy policy, but we are learning the Q function for the greedy policy meaning that after generating experience and training, we switch over to the greedy policy. The behaviour policy dictates how we act in the environment and the target policy is the policy we want to learn. Max action selection nature of the Q-learning update rule, leads us an optimal policy that converges more quickly. Our solution quickly updated the Q-matrix such that it consistently completed its task in 6 steps and gaining 66 rewards, which are the best best pair of results for completing the task quickly.

Interesting Unobserved Loop

Looking at the rewards structure, we thought that allowing the agent to receive + 20 reward for picking up the parcel and only getting a -10 punishment for dropping off the parcel at the wrong place would have resulted in the agent endlessly picking up and dropping off the parcel and getting a net +10 reward each time. Whilst the agent did achieve 84 rewards, which suggests that it gained an extra 20 rewards by picking up and dropping off the parcel twice, none of the episodes got stuck in such a loop and didn't give a reward higher that 84. This was of course only possible under reward structure A.

ADVANCED TASKS

7 DQN Implementation and Two Improvements

In the previous chapters we implemented SARSA and Q-Learning algorithms with a random search to show that how the agents' behaviour improves with learning algorithms, but these algorithms are tabular solutions to the Reinforcement Learning Problems and limited to low dimensional domains that can be fully observed or domains in which features can be handcrafted. However, in real world situation complexity, agents face high dimensional inputs and need to come up better action selection policies. Mnih et al. (2013) In this part, we apply a Deep Q Network, exploiting the recent developments in neural networks and its power of better handling of the high dimensional computations. With this approach, unlike tabular learning, we no longer need to map every possible state action pair to calculate Q-values.

Task and Domain

For this task we chose the Acrobot-v1 environment that provided by OpenAI Gym. Acrobot is a pendulum that has 2 links. Links swing freely and can pass by each other without collision. The pendulum described by two angles θ_1 and θ_2. First angle represented by θ_1 and the joint angle represented by θ_2. Environment has states that cosine, sine and the velocity of each angle. Possible actions are applying a rotational force of value +1, 0 and -1 on the joint. A position can be represented by θ_1 or θ_2 and finally with relative angle between these two angles $\theta_1 + \theta_2$ to the vertical position.

The task, on the other hand, to reach to the top point by taking a salvo(s) and hold that position as long as possible. Every episode limited to 100 movements. Agent gets a reward of +1 if the pendulum has the correct position or 0 otherwise.

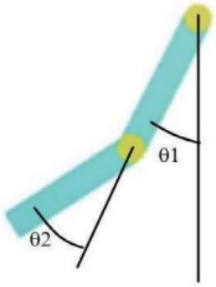


Figure 7: Acrobot Game

Key Concepts in DQN and Motivation for Improvements

Despite being computationally stronger, the utilisation of neural networks in reinforcement learning problems creates its own issues. Main problem here is that, is it possible to explain a reinforcement learning process as a supervised learning process to feed the neural network. Second and truly a much bigger question is that, whether or not we describe the process as a supervised leaning problem, how can we backpropagate this networks and update the weights, since the target (action) values always changing? At this point, another question to be answered is that what is the relation between the agent's feedback and the network?

Experience Replay

Data obtained from the reinforcement learning process is "online", meaning that observations or states that we feed the network are dependent to each other. If we think that the best action as a target value, which we want to predict, this predicted best action also changes depending on the incoming online observations from the environment. In calculations, for a single network, we simply pass S_t, then we pass S_{t+1} and obtain related Q-Values. By the way, we could not find any chance to backpropagate. Even if we use the same weights for S_{t+1} without doing a backprop, this will change the q* for each pass. This will lead to a very unstable training process, since our target q value is constantly changing, which issue referred as 'chasing your own tail'.

A fix to this issue, presented as a first improvement, using an experience replay buffer, the idea that was introduced back in 1992. An Experience Replay Buffer holds N steps of agents' feedback from the environment (experiences) and during the learning phase, the samples are drawn from this buffer uniformly at random, which eliminates the correlations between the samples used to train the neural network and gives independent and identically distributed samples. Lin (1992) In this way experience replay allows our network to generalise the different environment states and finally our policy can handle a variety of situations. There is evidence that a similar process takes place in animal brains. Animals appear to replay their past experiences in their hippocampus, which contributes to their learning. McClelland et al. (1995)

Duelling DQN Architecture

Another issue for the networks that, some actions taken by the agent has no effect on the environment, which makes the estimated action values meaningless because of the noise that actions created. Noise value makes DQN less effective to choose the maximal q values.

Fixing this problem, we are using the proposed Duelling DQN architecture, which simultaneously estimates the state value and the action advantages in parallel streams for a given state. The advantage value of an action in a given state is the additional expected cumulative reward that comes with choosing that action instead of what the policy in use suggests. This turns the computation of the choosing the action with the highest advantage is equivalent to choosing the action with the highest Q-value. Formula used in the code is as follows:

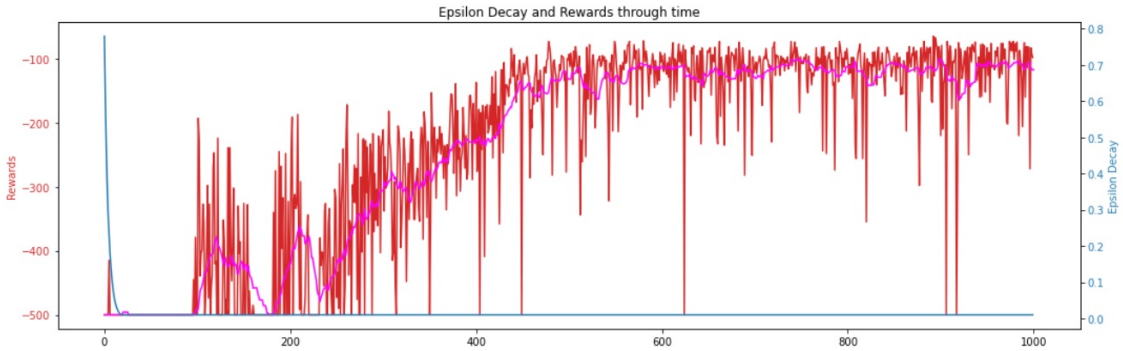


Figure 9: DQN Performance

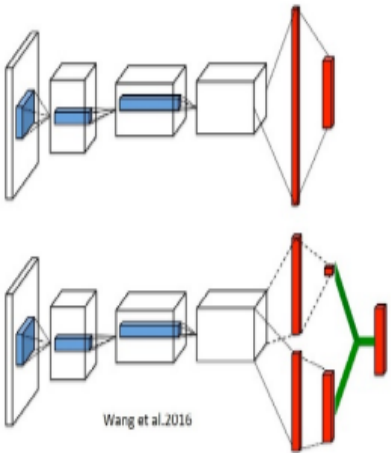


Figure 8: Duelling DQN Architecture

$$A_\pi(S,a) = Q_\pi(S,a) - V_\pi(S)$$

By obtaining the Q-values from the explicit representations of the state value and the action advantages, we enable the network to have a good representation of the state value without having to accurately estimate each action value for a given state. Intuitively, the duelling architecture can learn which states are (or are not) valuable, without having to learn the effect of each action for each state. This is particularly useful in states where its actions do not affect the environment in any relevant way. Wang et al. (2016)

8 Performance of the Deep Q Networks

For this section, 4 different network architectures have been used to experiment on the performance of the DQN with the Acrobot-v1 environment. First architecture is the vanilla DQN, then the performance of this architecture tried to be improved with an Experience Replay Buffer. The third algorithm which used is the Duelling DQN and then to improve its performance additional Experience Replay Buffer added to the algorithm. After countless of runs and experiments with different hyper-parameters have been applied, the Figure below shows the best performances which taken by the different architectures.

DQN

Figure 9 shows the best performance obtained from the Vanilla DQN architecture. Hyper-Parameters used are as follows with 1000 episode:

$\gamma = 0.9$	$\epsilon_{max} = 1.0$	$\epsilon_{min} = 0.01$
$\epsilon DecayRate = 0.9995$	Learning Rate = 0.0001	Number of Nodes in the Hidden Layer = 128

Experiments showed that the number of nodes in the hidden layers have the least effective on the convergence (finding the optimal poicy) of the neural network. Experiments run with 32, 64, 128 and 256 nodes on the hidden layer and it is observed that the minor effects on the learning and convergence by increasing complexity in the neural network architecture. On the other hand, most important parameters are the Max Epsilon and Epsilon Decay Rate. That is because the randomness

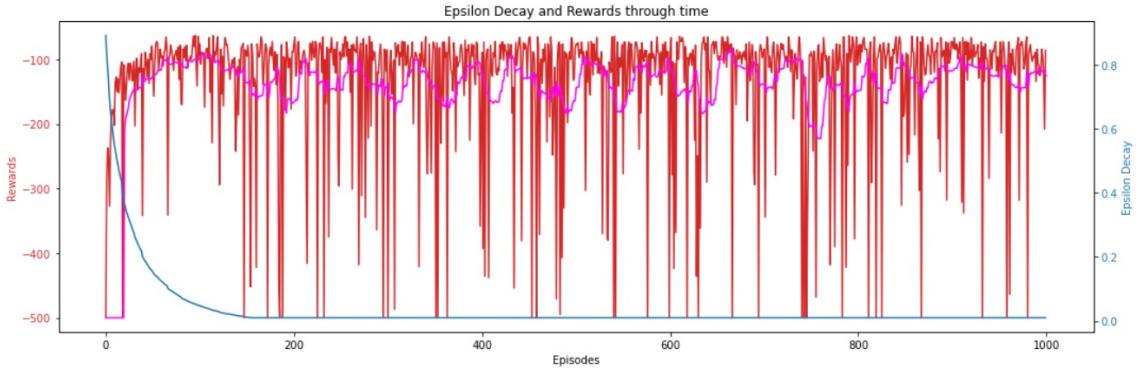


Figure 10: DQN + ERPerformance

in the actions taken cause fluctuations and these fluctuations (because of the non-stationary estimation of the outputs) delays the learning an optimal policy. Here, we did the epsilon decay so fast, so the randomness in the action selection did not have the effect of delay. More importantly it proved that the importance of the Independence of the events and Identically Distributed data are so important for the neural network architectures, because, Non-Stationary target values (Q-Value estimates and outputs) withheld the learning process and caused the fluctuations, meaning a non-optimal policy. Even with the tuned parameters, the figure above shows the fluctuations for more than 300 episodes and after that point DQN started to produce a near-optimal state-action selection strategy for the Acrobot-v1 environment. Additionally, high learning rate parameters (fast learning agent), (we used 0.1, 0.01, 0.001 and 0.0001), caused the same random fluctuations due to the gradient update in the loss function. Higher the updates caused high errors. Finally, Gamma value above or below 0.9 didn't performed in the agent's selection of the actions (above 0.9 led non-stable action selection, below 0.9 led relatively stable but non-optimal action-selection strategy).

DQN + Experience Replay

Figure 10 shows the best performance obtained from the ER Buffer added DQN architecture. Hyper-Parameters used are as follows with 1000 episode:

$\gamma = 0.99$	$\epsilon_{max} = 1.0$	$\epsilon_{min} = 0.01$
$\epsilon DecayRate = 0.999977$	Learning Rate = 0.001	Replay Size (Samples) = 200
Experience Length = 10000		Number of Nodes in the Hidden Layer = 128

Experiments showed that the number of nodes (32, 64, 128 and 256 nodes) in the hidden layers have the least effective on the convergence of the neural network. On the other hand, most important parameters are the Max Epsilon and Epsilon Decay Rate because of the replay buffer. Here, we first made the agent choose the random actions to explore the states and that gave the agent a reach experience buffer and accordingly network architecture converged so fast. More importantly it proved that the importance of the experience based action selection leads to more optimal policies and that the experience replay is a good solution and improved the policy. On the other hand, even with the tuned parameters, the figure above shows the fluctuations, indicating that the experience buffer size and the samples we withdrew are not sufficient enough for the environment. We think that these fluctuations caused by the uniformly driven samples, it is good to have a large experience buffer size but drive big enough sample sizes cause these fluctuations. Additionally, high learning rate parameters (fast learning agent), (we used 0.1, 0.01, 0.001 and 0.0001), are another reason for the same random fluctuations due to the gradient update in the loss function. Higher the updates caused high errors. Finally, Gamma value below 0.99 didn't performed in the agent's performance, gives us the conclusion that the experience replay does not require to look for immediate rewards because of the experience based action selection.

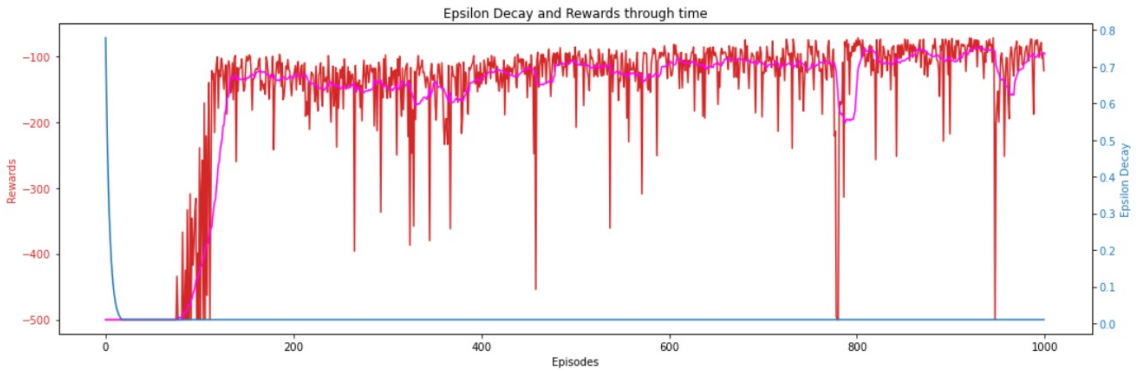


Figure 11: Duelling DQN Performance

Duelling DQN Architecture

Figure 11 shows the best performance obtained from the Duelling DQN architecture. Hyper-Parameters used are as follows with 1000 episode:

$\gamma = 0.99$	$\epsilon_{max} = 1.0$	$\epsilon_{min} = 0.01$
$\epsilon DecayRate = 0.9995$	Learning Rate = 0.0001	Number of Nodes in the Hidden Layer = 128

Experiments showed that the number of nodes (32, 64, 128 and 256 nodes) in the hidden layers have the least effective on the convergence of the neural network could be because of the parallel neural network structures working together. Indeed, we saw the Duelling DQN improvement due to the characteristic that choosing the action with the highest advantage is equivalent to choosing the action with the highest Q-value. This characteristic gave us less fluctuations (noise) and better convergence to an optimal policy. Max Epsilon and Epsilon Decay Rate showed its importance again to exploit the action selection ability of the duelling architecture. Here again, we did the epsilon decay so fast, so the randomness in the action selection did not have the effect of delay, so we could exploit the duelling dqn architecture. Even with the tuned parameters and Duelling DQN architecture, the figure above shows some fluctuations, we think that this could occur when an unfortunate sequence of action happens. Additionally, high learning rate parameters (fast learning agent), (we used 0.1, 0.01, 0.001 and 0.0001), caused the same random fluctuations due to the gradient update in the loss function. Higher the updates caused high errors. Finally, Gamma value below 0.99 didn't performed in the agent's performance, gives us the conclusion that, with the Duelling DQN characteristic of choosing the action with the highest Q-value, agent does not require to look for immediate rewards because of the Duelling DQN architecture.

Duelling DQN Architecture + Experience Replay

Figure 12 shows the best performance obtained from the ER added Duelling DQN architecture. Hyper-Parameters used are as follows with 1000 episode:

$\gamma = 0.99$	$\epsilon_{max} = 1.0$	$\epsilon_{min} = 0.01$
$\epsilon DecayRate = 0.9977$	Learning Rate = 0.001	Replay Size (Samples) = 200
Experience Length = 10000		Number of Nodes in the Hidden Layer = 128

Additional to the general comments that we provided, here we are seeing that the effect of the Duelling network max Q value selection and the effect of the learning from the experience with the replay buffer. Here, we first made the agent choose the random actions to explore the states, but because of the Duelling DQN characteristic network architecture led us a fast convergence. There are some fluctuations in the figure but at the second half of the training agent shows a stable and optimal policy. Additionally, high learning rate parameters (fast learning agent), (we used 0.1, 0.01, 0.001 and

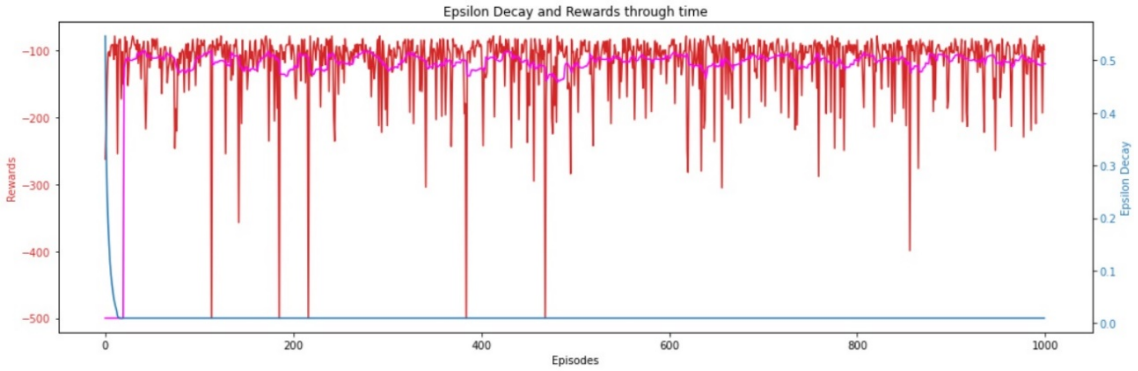


Figure 12: Duelling DQN + ER Performance

0.0001), caused the same random fluctuations due to the gradient update in the loss function. Higher the updates caused high errors. Finally, Gamma value below 0.99 did not performed in the agent's performance, gives us the conclusion that the Duelling architecture and experience replay combined, agent does not require to look for immediate rewards because of the experience based action selection.

9 RLLib Implementation of Double DQN

For task 9, I chose to run the Double DQN algorithm on the Atari Learning Environment game Pong. I chose to use the ram version of the game instead of the visual version for two reasons, first the input is extremely tiny at just 128 bytes *Pong-ram-v0* (2022) and second, I think it shows the nature of a (Double) Deep Q Network better than adding a ConvNet at the start. I say this because the ram version of the game remind us that the input to the algorithm is simply 128×8 bits of binary variables which is what the equation "sees" and updates its Q-Matrix with. I want to avoid the mistake of thinking that the algorithm can "sense" anything beyond numerical signals and "think" anything beyond computing the next update using the given equation.

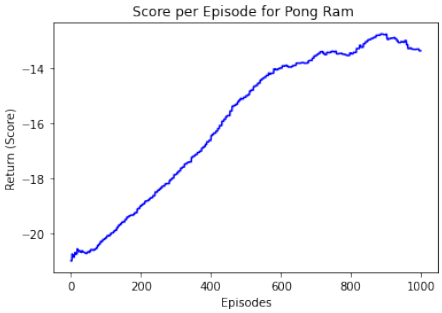


Figure 13: Rewards vs Episodes

9.1 Double DQN

A Double Deep Q-Network uses Double Q-Learning. The main benefit of using Double Q-Learning is to overcome the large overestimations which cause poor performance in Q-Learning algorithms when used in some stochastic environments. As stated by Hado Van Hasselt "These overestimations result from a positive bias that is introduced because Q-learning uses the maximum action value as an approximation for the maximum expected action value." Van Hasselt (2010) The pong game has stochasticity because it is not known how the opponent's bat will move and hit the ball or whether it will hit the ball at all. The Double DQN achieves this by evaluating the greedy policy with respect to the online network, and uses the target network to estimate its value.

For the evaluation, the weights of the second network are replaced with the target network's weights. This is usually done less frequently than every episode, roughly every 10 episodes, in order to solve the problem with constantly moving targets.

The reason I chose to use the Double DQN is purely based on my initial tuning work with *ray.tune* and seeing the greatest improvement in the final pong score when using Double DQN as opposed to not using it.

10 Double DQN - Results and Analysis

The Atari game of pong ends when either of the two players gets 21 points, in this investigation I am plotting the points that my agent receives and to simplify the score board, i.e. instead of having two scores, 1 for each player, I'll only present the net score here. So a score of -21 is a shameful loss for our agent (Agent scores 0 and the other player scores 21) and all the way up to + 21 would be a win with 21 scores and no points lost for our agent.

In order to get the result that I'm showing in figure 13, I ran a $1,000$ episodes of the game using the double_q option in the *ray.rllib.agents.dqn* package.

I decided which algorithms to enable based on the output I got from *ray.tune*. In my first set of tuning tests switching dueling off and keeping gamma at 0.9 instead of 0.99 resulted in a final score of -16 instead of three other scores between -19 to -21.

Switching double_q on resulted in -17 instead of -19 scores so I kept double_q.

Next, I tested 4 learning rates from 0.1 to 0.0001 (My CPU only has 4 threads so I could only do 4 at a time if I didn't want to wait till the end to move on to the next set of pending tests). The learning rate (LR) 0.001 edged ahead of the others by one to two scores.

Lastly I tested Prioritised Experience Replay (PER) and both with it and without I got scores closely in between -18 and -19.

In the end I settled for double_q and PER enabled, gamma = 0.9 and LR at 0.001. Over a $1,000$ episodes of the game the agent rather smoothly went from -21 score (the shameful loss) to a less shameful -13.3 scores at its best. Towards the end of the training the rate of increase in the score slowed down and tended to fluctuate a little bit. This is likely a point where the past learning of the agent was starting to lack improving the score and it needed to learn new tricks, perhaps bouncing the ball off the wall in a difficult manner for its opponent. These ideas could be confirmed in further studies by passing the game logs through an emulator and manually noting the types of tricks the agent has learnt. I haven't attempted this but I would expect the first era of improvement from about -20 to -15 simply be due to having the bat in the middle of where the ball is and the agent will later learn how to angle the ball to be able to actually play some winning episodes.

11 Extra Task – PPO Implementation

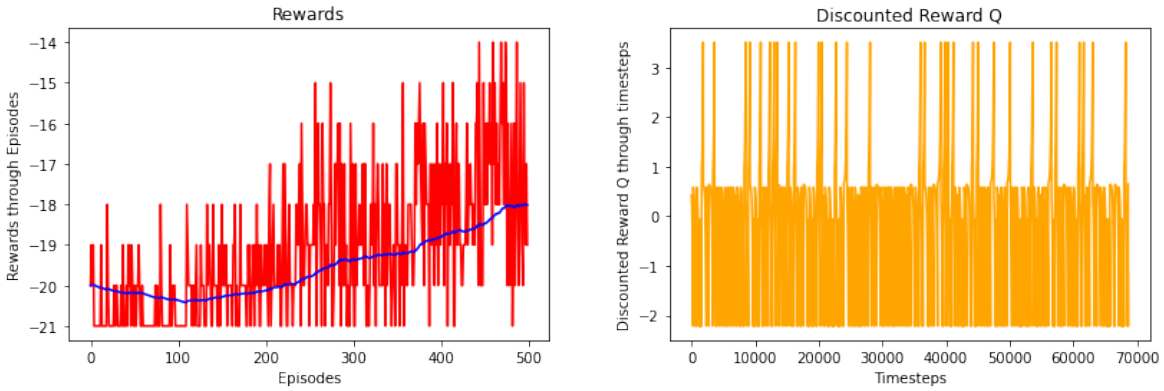
For task 11 we decided to work on an implementation of the PPO algorithm. In previous tasks, the algorithms used were all Value Base Methods. In the general Q-Learning approach, a policy is obtained by trying and learning from a mapping from action space to state space and comparing that with different policies to find an optimal one. Policy Based Methods has a direct approach on the solution to obtain an optimal policy. Kim (2021) For this task, Environment is chosen to be the Pong (NoFrameSkip-v4).

In PPO the policy estimation takes the observations from the environment as an input and outputs a suggestion for an actions to take. The advantage is defined as the estimation of the relative value of the selected action for the current state.

The advantage is calculated as a discounted reward (Q) — value function. The value function simply provides an estimate of the discounted rewards' sum. Kim (2021) The problem with the PPO is that the value estimate will be very noisy due to the variance caused by the network, as can be seen in figure 14.

Multiplying the log probabilities of the policy's output and advantage function gives us a clever optimisation function. If the advantage is greater than zero, meaning the agent got better than average returns for the actions it took in the sample trajectory, the policy gradient would be positive and increase the probability of picking these actions again when we encounter a similar state. If the advantage is less than zero, the policy gradient would be negative to do the exact opposite. Figure 14 shows the rewards over time, 14 (a), and discounted reward(Q), 14 (b) .

Figure 14: Rewards and Discounted Rewards for PPO



(a) Reward per Episode

(b) Discounted Reward per Time Step

References

Kim, T. (2021), 'Understanding proximal policy optimization'.
URL: *https://towardsdatascience.com/understanding-and-implementing-proximal-policy-optimization-schulman-et-al-2017-9523078521ce*

Lin, L.-J. (1992), 'Self-improving reactive agents based on reinforcement learning, planning and teaching.'.

McClelland, J. L., McNaughton, B. L. & O'Reilly, R. C. (1995), 'Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory'.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. & Riedmiller, M. (2013), 'Playing atari with deep reinforcement learning.'.

OcadoGroup (2022), 'Technology behind the ocado smart platform'.
URL: *https://www.ocadogroup.com/technology/powering-osp/*

Pong-ram-v0 (2022).
URL: *https://gym.openai.com/envs/Pong-ram-v0/*

Sutton, R. S. & Barto, A. G. (2018), *2.3*, The MIT Press.

Van Hasselt, H. (2010), 'Double q-learning.'.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M. & Freitas, N. (2016), 'Dueling network architectures for deep reinforcement learning'.

Appendices

Contributions to the team tasks by Abdulaziz Aksozlu

Together with Mali Colak, we decided the environment to use, he suggested the warehouse idea and I added the Ocado Smart Platform idea. We searched for grids used to do RL on and found a Python course by learnpythonwithrune.org which had very helpful articles and we used that as the basis for our basic part warehouse class. After jointly building the class I did further work on optimising for the best hyperparameters.

For task 7 we studied together in the library and found Phil Tabor's GitHub repository Deep Q-Learning Paper To Code on DQNs very helpful. (https://github.com/philtabor/Deep-Q-Learning-Paper-To-Code) We coded a similar vanilla DQN using pytorch and implemented our Experience replay and duelling networks together.

For task 11 we found an RLLib implementation of PPO on my Atari game Pong and mostly used that for our coursework. (http://www.sagargv.com/blog/pong-ppo/)

Basic Task

```
# Python notes used from https://www.learnpythonwithrune.org/

# Q-Learning notes used from https://deeplizard.com/learn/video/nyjbcRQ-uQ8

from IPython import display
path = "./Warehouse_Layout.png" # The image Warehouse_Layout.png must be in the same directory as this notebook
print(" *** \n Here is a representation of our warehouse used in this investigation \n *** \n ")
display.Image(path)

import numpy as np
from scipy.interpolate import make_interp_spline
import random
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="dark")
sns.set_context("talk")
sns.color_palette("dark")
%matplotlib inline
```

```
class Warehouse:
    def __init__(self, size, pickup, drop_off, position, kiosk):       # We will initialize a Warehouse object
        self.size = size                                               # The warehouse is a square grid, dimensions size X size
        self.pickup = pickup                                           # Pickup is (x, y) for the ordered product
        self.drop_off = drop_off                                       # Drop off is always the target for the product to be dropped off at
        self.position = position                                       # Position is where the agent robot currently is, it starts from an initial position
        self.kiosk = kiosk                                             # kiosk location to avoid in warehouse
        self.cage_full = False                                         # Boolean to track if the ordered product is in the cage or not

    def get_number_of_states(self):
        return self.size**5                                                # Compute a number of states for later use in making Q-Table
```

```
                                                                # All the states could be enumerated 1 by 1, but this seems simpler to implement
                                                                # SAMPLE STATE NUMBER - [position[0], position[1], pickup[0], pickup[1], cage_full]
    def get_state(self):
        state = self.position[0]*self.size**4                   # Multiply agent location coordinates by size
        state = state + self.position[1]*self.size**3           # to get a value for current state
        state = state + self.pickup[0]*self.size**2             # Also multiply by the pickup position to capture that info as well
        state = state + self.pickup[1]*self.size
        if self.cage_full:                                      # Add 1 if the agent has picked up the ordered product
            state = state + 1
        return state
                                                                # The main idea here is to number the states in a way to give each coordinate value a different order
                                                                # of magnitude so that states never have the same number. Then add 1 if the cage is full.
                                                                # Again, there are methods that could return state numbers that are more compact,
                                                                # but this is easier and works just fine.

    def take_action(self, action):
        bump_wall =             -10
        bump_kiosk =            -20
        step =                  -1
        failed_pickup =         -10
        failed_drop_off =       -10
        successful_pick_up =    20
        successful_drop_off =   50
        (x, y) = self.position
        if action == 0:                                         # Go up
            if y == self.size - 1:                              # Check if agent is at top boundary
```

```
                return bump_wall, False                                  # Return reward and whether episode is over or not (Punish for trying to go out of warehouse)
            elif y == self.kiosk[1]-1:                                   # Check if agent is trying to move into kiosk and if so, punish
                return bump_kiosk, False
            else:                                                        # If agent isn't at the boundary...
                self.position = (x, y + 1)                               # Make the appropriate change to position coordinates
                return step, False                                         # Every time step returns -1 reward to discourage wandering
        elif action == 1:                                                # Go down
            if y == 0:                                                   # Check if agent is at bottom boundary
                return bump_wall, False
            elif y == self.kiosk[1]+1:
                return bump_kiosk, False
            else:
                self.position = (x, y - 1)
                return step, False
        elif action == 2:                                                # Go left
            if x == 0:
                return bump_wall, False
            elif y == self.kiosk[0]+1:
                return bump_kiosk, False
            else:
                self.position = (x - 1, y)
                return step, False
        elif action == 3:                                                # Go right
            if x == self.size - 1:
                return bump_wall, False
            elif y == self.kiosk[0]-1:
                return bump_kiosk, False
            else:
                self.position = (x + 1, y)
                return step, False
        elif action == 4:                                                # Pickup item
            if self.cage_full:                                           # Punish if agent tries to pick up whilst cage is full
                return failed_pickup, False
            elif self.pickup != (x, y):                                  # If current (x,y) != pick up location, then punish
                return failed_pickup, False
            else:
                self.cage_full = True                                    # else, the agent picked up the ordered product and gets 20 reward
                return successful_pick_up, False
        elif action == 5:                                                # Drop off item
            if not self.cage_full:                                       # Punish for trying to drop off from empty cae
                return failed_drop_off, False
            elif self.drop_off != (x, y):                                # If agent drops off at a position that isn't the drop-off location
                self.pickup = (x, y)                                     # Update the pickup location  to the current location
                self.cage_full = False                                   # Update that the cage is now empty
                return failed_drop_off, False                             # Punish for wrong drop off
            else:
```

```
                return successful_drop_off, True                # Voilà! If all above are false, then the agent has accomplished its mission
```

```
def random_search( size, pickup, drop_off, position, kiosk):
    warehouse = Warehouse(size, pickup, drop_off, position, kiosk)

    done = False
    steps = 0
    rewards = 0
    while not done:
        action = random.randint(0, 5)
        reward, done = warehouse.take_action(action)
        steps += 1
        rewards = rewards + reward
        print(".     Current position is ", warehouse.position, ".    Action is ", action, ".  Current step is ", steps)

    return steps, rewards
```

```
def take_E_greedy_action(state, epsilon):
    if epsilon > random.uniform(0, 1):
        action = random.randint(0, 5)
    else:
        action = np.argmax(q_table[state])
    return action

def q_learning(episodes = 1000, gamma = 0.9, alpha = 0.5, epsilon = 1, epsilon_decay_rate = 0.999):

    cache_q = np.array([[0 ,0 , 0]])

    for episode in range(episodes):
        warehouse = Warehouse(5, (1, 1), (2, 2), (0, 0), (3, 3))
        done = False
        steps = 0
        rewards = 0

        while not done:
            state = warehouse.get_state()
            action = take_E_greedy_action(state, epsilon)


            if epsilon > 0.01:
                epsilon *= epsilon_decay_rate
            reward, done = warehouse.take_action(action)
            steps += 1
            rewards = rewards + reward
            new_state = warehouse.get_state()
            new_state_max = np.max(q_table[new_state])

            q_table[state, action] = (1-alpha) * q_table[state, action]  +  alpha * (reward + gamma * new_state_max - q_table[state, action])
        if episode % 10 == 0:
            print("Episode: ", episode, "   Step:", steps, " Epsilon is ", epsilon, "Return is ", rewards)
        cache_q =  np.append(cache_q, [[episode, steps, rewards]], axis = 0)

    return cache_q
```

```python
def sarsa(episodes = 200, gamma = 0.9, alpha = 0.5, epsilon = 1, epsilon_decay_rate = 0.999):

    cache_sarsa = np.array([[0 ,0 , 0]])

    for episode in range(episodes):

        warehouse = Warehouse(5, (1, 1), (2, 2), (0, 0), (3, 3))
        done = False
        steps = 0
        rewards = 0
        if epsilon > 0.01:
            epsilon *= epsilon_decay_rate

        state = warehouse.get_state()
        action = take_E_greedy_action(state, epsilon)
        while not done:
            #state = warehouse.get_state()
            reward, done = warehouse.take_action(action)
            steps += 1
            rewards = rewards + reward
            new_state = warehouse.get_state()

            # Take greedy action again for sarsa update
            new_action = take_E_greedy_action(state, epsilon)

            q_table[state, action] = (1-alpha)*q_table[state, action]  +  alpha * (reward + gamma * q_table[new_state, new_action] - q_table[state, action])
            state = new_state
            action = new_action
        if episode % 10 == 0:
            print("Episode: ", episode, "   Step:", steps, " Epsilon is ", epsilon, "Return is ", rewards)
        cache_sarsa =  np.append(cache_sarsa, [[episode, steps, rewards]], axis = 0)

    return cache_sarsa
```

```python
# Data prep before plotting
x_sarsa_episodes = cache_sarsa[:, 0]
y_sarsa_steps = cache_sarsa[:, 1]
y_sarsa_return = cache_sarsa[:, 2]


plt.title("SARSA Performance, (Episodes vs steps)")
plt.xlabel("Episode")
plt.ylabel("Steps")
plt.plot(x_sarsa_episodes, y_sarsa_steps, color ="red", label = "Steps")
plt.legend(loc='upper right')
plt.show()


plt.title("SARSA Performance, (Episodes vs Return)")
plt.xlabel("Episode")
plt.ylabel("Return")
plt.plot(x_sarsa_episodes, y_sarsa_return, color ="blue", label = "Return")
plt.legend(loc='lower right')
plt.show()


```

```
# Data prepping before plotting
x_q_episodes =  cache_q[10:, 0]
y_q_steps =     cache_q[10:, 1]
y_q_return =    cache_q[10:, 2]

plt.ylim((5, 30))

plt.title("Q_Learning Performance, (Episodes vs steps) \n Alpha & Gamma = 0.9  - Epsilon Decay Rate = 0.95 \n Reward Set A \n ")
plt.xlabel("Episode")
plt.ylabel("Steps")
plt.plot(x_q_episodes, y_q_steps, color ="red")
plt.show()

plt.ylim((-100, 100))
plt.title("Q_Learning Performance, (Episodes vs Return) \n  Alpha & Gamma = 0.9  - Epsilon Decay Rate = 0.95 \n Reward Set A \n")
plt.xlabel("Episode")
plt.ylabel("Return")
plt.plot(x_q_episodes, y_q_return, color ="blue")
plt.show()


cache_q_alpha01 = q_learning(episodes = 1000, gamma = 0.7, alpha = 0.1, epsilon = 1, epsilon_decay_rate = 0.99999)
cache_q_alpha02 = q_learning(episodes = 1000, gamma = 0.7, alpha = 0.2, epsilon = 1, epsilon_decay_rate = 0.99999)
cache_q_alpha04 = q_learning(episodes = 1000, gamma = 0.7, alpha = 0.4, epsilon = 1, epsilon_decay_rate = 0.99999)
cache_q_alpha06 = q_learning(episodes = 1000, gamma = 0.7, alpha = 0.6, epsilon = 1, epsilon_decay_rate = 0.99999)
cache_q_alpha09 = q_learning(episodes = 1000, gamma = 0.7, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.99999)


plt.figure(figsize= (12,8), dpi = 150)
plt.title("Q_Learning Performance, (Episodes vs steps) \n "
          "gamma = 0.7, epsilon = 1, epsilon_decay_rate = 0.99999) \n"
          "First 200 episodes not shown to focus on later stages")

plt.xlabel("Episode")
plt.ylabel("Steps")

plt.plot(cache_q_alpha01[200:,0], cache_q_alpha01[200:,1],  color='r',  label='Alpha = 0.1', alpha = 0.9)
#plt.plot(cache_q_alpha02[200:,0], cache_q_alpha02[200:,1],  color='',  label='Alpha = 0.2', alpha = 0.7)
plt.plot(cache_q_alpha04[200:,0], cache_q_alpha04[200:,1],  color='b',  label='Alpha = 0.4', alpha = 0.5)
#plt.plot(cache_q_alpha06[200:,0], cache_q_alpha06[200:,1],  color='',  label='Alpha = 0.6', alpha = 0.25)
plt.plot(cache_q_alpha09[200:,0], cache_q_alpha09[200:,1],  color='k',  label='Alpha = 0.9', alpha = 0.1)
plt.legend(loc='upper right')
plt.show()
"""plt.plot(cache_q_alpha01[200:,0], cache_q_alpha01[200:,1],  color='#ed0e0e', label='Alpha = 0.1', alpha = 0.9)
plt.plot(cache_q_alpha02[200:,0], cache_q_alpha02[200:,1],  color='#e65555', label='Alpha = 0.2', alpha = 0.7)
plt.plot(cache_q_alpha04[200:,0], cache_q_alpha04[200:,1],  color='#bd6464', label='Alpha = 0.4', alpha = 0.5)
```

```
plt.plot(cache_q_alpha06[200:,0], cache_q_alpha06[200:,1],  color='#c99191',  label='Alpha = 0.6', alpha = 0.25)
plt.plot(cache_q_alpha09[200:,0], cache_q_alpha09[200:,1],  color='#d6b4b4',  label='Alpha = 0.9', alpha = 0.1)
"""

cache_q_gamma01 = q_learning(episodes = 1000, gamma = 0.1, alpha = 0.1, epsilon = 1,  epsilon_decay_rate = 0.99999)
cache_q_gamma02 = q_learning(episodes = 1000, gamma = 0.2, alpha = 0.1, epsilon = 1,  epsilon_decay_rate = 0.99999)
cache_q_gamma04 = q_learning(episodes = 1000, gamma = 0.4, alpha = 0.1, epsilon = 1,  epsilon_decay_rate = 0.99999)
cache_q_gamma06 = q_learning(episodes = 1000, gamma = 0.6, alpha = 0.1, epsilon = 1,  epsilon_decay_rate = 0.99999)
cache_q_gamma09 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.1, epsilon = 1,  epsilon_decay_rate = 0.99999)

plt.figure(figsize= (12,8), dpi = 150)
plt.title("Q_Learning Performance, (Episodes vs steps) \n "
          "Alpha = 0.1, epsilon = 1, epsilon_decay_rate = 0.99999) \n"
          "First 200 episodes not shown to focus on later stages")

plt.xlabel("Episode")
plt.ylabel("Steps")

plt.plot(cache_q_gamma01[200:,0], cache_q_gamma01[200:,1],  color='r',  label='Gamma = 0.2', alpha=0.9)
#plt.plot(cache_q_gamma02[200:,0], cache_q_gamma02[200:,1],  color='g',  label='Gamma = 0.2', alpha=0.6)
plt.plot(cache_q_gamma04[200:,0], cache_q_gamma04[200:,1],  color='b',  label='Gamma = 0.4', alpha=0.4)
#plt.plot(cache_q_gamma06[200:,0], cache_q_gamma06[200:,1],  color='m',  label='Gamma = 0.6', alpha=0.2)
plt.plot(cache_q_gamma09[200:,0], cache_q_gamma09[200:,1],  color='k',  label='Gamma = 0.9', alpha=0.3)
plt.legend(loc='upper right')
plt.show()


plt.figure(figsize= (12,8), dpi = 150)
plt.title("Q_Learning Performance, (Episodes vs steps) \n "
          "Alpha = 0.1, epsilon = 1, epsilon_decay_rate = 0.99999) \n"
          "Last 50 episodes shown to focus on the final stage")

plt.xlabel("Episode")
plt.ylabel("Steps")

plt.plot(cache_q_gamma01[550:,0], cache_q_gamma01[550:,1],  color='r',  label='Gamma = 0.2')
#plt.plot(cache_q_gamma02[550:,0], cache_q_gamma02[550:,1],  color='#e65555',  label='Gamma = 0.2')
plt.plot(cache_q_gamma04[550:,0], cache_q_gamma04[550:,1],  color='g',  label='Gamma = 0.4')
#plt.plot(cache_q_gamma06[550:,0], cache_q_gamma06[550:,1],  color='#c99191',  label='Gamma = 0.6')
plt.plot(cache_q_gamma09[550:,0], cache_q_gamma09[550:,1],  color='b',  label='Gamma = 0.9')
plt.legend(loc='upper right')
plt.show()

```

```
#cache_q_E_Decay1 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.9)
#cache_q_E_Decay2 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.99)
#cache_q_E_Decay3 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.999)
#cache_q_E_Decay4 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.99999)
#cache_q_E_Decay5 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.99999)
cache_q_E_Decay6 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.92)
cache_q_E_Decay7 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.95)
cache_q_E_Decay8 = q_learning(episodes = 1000, gamma = 0.9, alpha = 0.9, epsilon = 1, epsilon_decay_rate = 0.98)

plt.figure(figsize= (12,8), dpi = 150)
plt.title("Q_Learning Performance, (Episodes vs steps) \n "
          "Alpha = 0.9, gamme = 0.9, epsilon = 1) \n"
          "First 200 episodes not shown to focus on later stages \n"
          "Only step values between 5 and 10 are shown since the best solution is 6 steps")

plt.xlabel("Episode")
plt.ylabel("Steps")

plt.ylim((5, 10))

plt.plot(cache_q_E_Decay1[200:,0], cache_q_E_Decay1[200:,1],  color='r',  label='E_Decay = 0.9', alpha=0.9)
plt.plot(cache_q_E_Decay2[200:,0], cache_q_E_Decay2[200:,1],  color='g',  label='E_Decay = 0.99', alpha=0.6)
plt.plot(cache_q_E_Decay3[200:,0], cache_q_E_Decay3[200:,1],  color='b',  label='E_Decay = 0.999', alpha=0.4)
plt.plot(cache_q_E_Decay4[200:,0], cache_q_E_Decay4[200:,1],  color='m',  label='E_Decay = 0.9999', alpha=0.2)
plt.plot(cache_q_E_Decay5[200:,0], cache_q_E_Decay5[200:,1],  color='k',  label='E_Decay = 0.99999', alpha=0.3)
plt.legend(loc='upper right')
plt.show()


plt.figure(figsize= (12,8), dpi = 150)
plt.title("Q_Learning Performance, (Episodes vs steps) \n "
          "Alpha = 0.9, gamme = 0.9, epsilon = 1) \n"
          "First 200 episodes not shown to focus on later stages \n"
          "Only step values between 6 and 8 are shown since the best solution is 6 steps")

plt.xlabel("Episode")
plt.ylabel("Steps")

plt.ylim((6, 8))

plt.plot(cache_q_E_Decay6[200:,0], cache_q_E_Decay6[200:,1],  color='r',  label='E_Decay = 0.92', alpha=0.9)
plt.plot(cache_q_E_Decay7[200:,0], cache_q_E_Decay7[200:,1],  color='g',  label='E_Decay = 0.95', alpha=0.6)
plt.plot(cache_q_E_Decay8[200:,0], cache_q_E_Decay8[200:,1],  color='b',  label='E_Decay = 0.98', alpha=0.4)
```

```

plt.legend(loc='upper right')
plt.show()
```

Advanced Task

```
class DQN(nn.Module):
    def __init__(self,lr,n_actions,n_states):
        super(DQN,self).__init__()

        self.fc1 = nn.Linear(*n_states,hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden,n_actions)

        self.optimizer = optim.Adam(self.parameters(),lr=lr)
        self.loss = nn.MSELoss()
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')
        self.to(self.device)

    def forward(self,state):
        layer1 = F.relu(self.fc1(state))
        actions = self.fc2(layer1)

        return actions


class DQNAgent():
    def __init__(self):

        self.lr = lr
        self.n_states = n_states
        self.n_actions = n_actions
        self.gamma = gamma
        self.epsilon = epsilon
        self.eps_dec = eps_dec
        self.eps_min = eps_min
        self.action_space = [i for i in range(self.n_actions)]

        self.Q = DQN(self.lr, self.n_actions, self.n_states)

    def epsilon_greedy_action_selection(self, observation):
        if np.random.random() > self.epsilon:
            state = T.tensor(observation, dtype=T.float).to(self.Q.device)
            actions = self.Q.forward(state)
            action = T.argmax(actions).item()
        else:
            action = np.random.choice(self.action_space)

        return action

    def epsilon_update(self):
        self.epsilon = self.epsilon * self.eps_dec \
                        if self.epsilon > self.eps_min else self.eps_min

    def learn(self, state, action, reward, state_):
        self.Q.optimizer.zero_grad()
        states = T.tensor(state, dtype=T.float).to(self.Q.device)
        actions = T.tensor(action).to(self.Q.device)
        rewards = T.tensor(reward).to(self.Q.device)
```

```
        states_ = T.tensor(state_, dtype=T.float).to(self.Q.device)

        q_pred = self.Q.forward(states)[actions]

        q_next = self.Q.forward(states_).max()

        q_target = rewards + self.gamma * q_next

        loss = self.Q.loss(q_target, q_pred).to(self.Q.device)
        loss.backward()
        self.Q.optimizer.step()
        self.epsilon_update()

#Testing the Agent in the CartPole Environment
env = gym.make('Acrobot-v1')
n_states = env.observation_space.shape
n_actions = env.action_space.n

gamma = 0.99
epsilon = 1.0
eps_dec = 0.9995
eps_min = 0.01

hidden = 128
lr = 0.0001

n_games = 1000


scores = []
eps_history = []


dqnAgent = DQNAgent()

for i in range(n_games):
    score = 0
    done = False
    obs = env.reset()

    while not done:
        action = dqnAgent.epsilon_greedy_action_selection(obs)
        obs_, reward, done, info = env.step(action)
        score += reward
        dqnAgent.learn(obs, action, reward, obs_)
        obs = obs_
    scores.append(score)
    eps_history.append(dqnAgent.epsilon)

    if i % 100 == 0:
        avg_score = np.mean(scores[-100:])
        #print('episode ', i, 'score %.1f avg score %.1f epsilon %.2f' % (score,
        ↪  avg_score, agent.epsilon))
```

```
#Calculating the running mean for a smooth visualization of the rewards
#Here we are calculating the every 20 episode for smoothing
window_size = 20
i = 0
# Initialize an empty list to store moving averages
moving_averages = [-500]*window_size
```

```

while i < len(scores) - window_size + 1:

    # Calculate the average of current window
    window_average = round(np.sum(scores[i:i+window_size]) / window_size, 2)

    # Store the average of current
    # window in moving average list
    moving_averages.append(window_average)

    # Shift window to right by one position
    i += 1


#Plots to see the decay in the epsilon and rewards that agent collected
fig,ax1 = plt.subplots(figsize = (15,5))
plt.title('Epsilon Decay and Rewards through time')

color = 'tab:red'
ax1.set_xlabel('Episodes')
ax1.set_ylabel('Rewards', color=color)
ax1.plot(scores, color=color)
ax1.plot(moving_averages, color='magenta')
ax1.tick_params(axis='y', labelcolor=color)

ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis
color = 'tab:blue'
ax2.set_ylabel('Epsilon Decay', color=color)  # we already handled the x-label with ax1
ax2.plot(eps_history, color=color)
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()  # otherwise the right y-label is slightly clipped
plt.show()
```

```
class DQNExpRep(nn.Module):
    def __init__(self,lr,n_actions,n_states):
        super(DQNExpRep,self).__init__()

        self.fc1 = nn.Linear(*n_states,hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden,n_actions)

        self.optimizer = optim.Adam(self.parameters(),lr=lr)
        self.loss = nn.MSELoss()
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')
        self.to(self.device)

    def forward(self,state):
        layer1 = F.relu(self.fc1(state))
        actions = self.fc2(layer1)

        return actions


class DQNExpRepAgent():
    def __init__(self):

        self.lr = lr
        self.n_states = n_states
        self.n_actions = n_actions
```

```
        self.gamma = gamma
        self.epsilon = epsilon
        self.eps_dec = eps_dec
        self.eps_min = eps_min
        self.action_space = [i for i in range(self.n_actions)]

        self.Q = DQNExpRep(self.lr, self.n_actions, self.n_states)

    def epsilon_greedy_action_selection(self, observation):
        if np.random.random() > self.epsilon:
            state = T.tensor(observation, dtype=T.float).to(self.Q.device)
            actions = self.Q.forward(state)
            action = T.argmax(actions).item()
        else:
            action = np.random.choice(self.action_space)

        return action

    def epsilon_update(self):
        self.epsilon = self.epsilon * self.eps_dec \
                           if self.epsilon > self.eps_min else self.eps_min


    def learn_from_exp(self, state, action, reward, state_, memory, replay_size, gamma):
        if len(memory) >= replay_size:

            experience = random.sample(memory, replay_size)


            predictions = []
            targets = []
            for state, action, reward, state_, done in experience:

                states = T.tensor(state, dtype=T.float).to(self.Q.device)
                actions = T.tensor(action).to(self.Q.device)
                rewards = T.tensor(reward).to(self.Q.device)
                states_ = T.tensor(state_, dtype=T.float).to(self.Q.device)
                done = T.tensor(state_, dtype=T.bool).to(self.Q.device)

                predictions.append(state)
                q_pred = self.Q.forward(predictions).tolist()

                if done:
                    #q_target = q_pred
                    q_target = q_pred.max()
                else:
                    q_next = self.Q.forward(states_).max()
                    q_target = reward + self.gamma * q_next

                targets.append(q_target)
                q_target = self.Q.forward(targets).tolist()


            self.Q.optimizer.zero_grad()

            loss = self.Q.loss(targets, states).to(self.Q.device)
            loss.backward()
            self.Q.optimizer.step()
        self.epsilon_update()


```

```
#Testing the Agent in the CartPole Environment
env = gym.make('Acrobot-v1')
n_states = env.observation_space.shape
n_actions = env.action_space.n


gamma = 0.9
epsilon = 1.0
eps_dec = 0.99977
eps_min = 0.01

hidden = 128
lr = 0.001

n_games = 1000


scores = []
eps_history = []


agent = DQNExpRepAgent()
memory = deque(maxlen=50)
replay_size = 20



for i in range(n_games):
    score = 0
    done = False
    obs = env.reset()

    while not done:
        action = agent.epsilon_greedy_action_selection(obs)
        obs_, reward, done, info = env.step(action)
        score += reward
        agent.learn_from_exp(obs, action, reward, obs_,memory, replay_size, gamma)
        obs = obs_
    scores.append(score)
    eps_history.append(agent.epsilon)

    if i % 100 == 0:
        avg_score = np.mean(scores[-100:])
        #print('episode ', i, 'score %.1f avg score %.1f epsilon %.2f' % (score,
        ↪  avg_score, agent.epsilon))
```

```
#Calculating the running mean for a smooth visualization of the rewards
#Here we are calculating the every 20 episode for smoothing
window_size = 20
i = 0
# Initialize an empty list to store moving averages
moving_averages = [-500]*window_size

while i < len(scores) - window_size + 1:

    # Calculate the average of current window
    window_average = round(np.sum(scores[i:i+window_size]) / window_size, 2)

    # Store the average of current
    # window in moving average list
```

```
    moving_averages.append(window_average)

    # Shift window to right by one position
    i += 1


#Plots to see the decay in the epsilon and rewards that agent collected
fig,ax1 = plt.subplots(figsize = (15,5))
plt.title('Epsilon Decay and Rewards through time')

color = 'tab:red'
ax1.set_xlabel('Episodes')
ax1.set_ylabel('Rewards', color=color)
ax1.plot(scores, color=color)
ax1.plot(moving_averages, color='magenta')
ax1.tick_params(axis='y', labelcolor=color)

ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis
color = 'tab:blue'
ax2.set_ylabel('Epsilon Decay', color=color)  # we already handled the x-label with ax1
ax2.plot(eps_history, color=color)
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()  # otherwise the right y-label is slightly clipped
plt.show()
```

```
class DuelingModel(nn.Module):
    def __init__(self,lr,n_actions,n_states):

        super(DuelingModel, self).__init__()
        #Initializing the dueling structure
        #State-Dependent action advantages stream
        self.adv1 = nn.Linear(*n_states, hidden)
        self.adv2 = nn.Linear(hidden, n_actions)
        #State-Value stream
        self.val1 = nn.Linear(*n_states, hidden)
        self.val2 = nn.Linear(hidden, 1)

        self.optimizer = optim.Adam(self.parameters(),lr=lr)
        self.loss = nn.MSELoss()
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')
        self.to(self.device)

    def forward(self,state):
        #Forward propagation with dueling architecture
        adv = nn.functional.relu(self.adv1(state))
        adv = self.adv2(adv)
        val = nn.functional.relu(self.val1(state))
        val = self.val2(val)

        actions = val + adv - adv.mean()
        #The Advantage is a quantity that obtained by subtracting the Q-value, by the
        ↪  V-value:
        return  actions


class DuelingModelAgent():
    def __init__(self):

        self.lr = lr
```

```
        self.n_states = n_states
        self.n_actions = n_actions
        self.gamma = gamma
        self.epsilon = epsilon
        self.eps_dec = eps_dec
        self.eps_min = eps_min
        self.action_space = [i for i in range(self.n_actions)]

        self.Q = DuelingModel(self.lr, self.n_actions, self.n_states)

    def epsilon_greedy_action_selection(self, observation):
        if np.random.random() > self.epsilon:
            state = T.tensor(observation, dtype=T.float).to(self.Q.device)
            actions = self.Q.forward(state)
            action = T.argmax(actions).item()
        else:
            action = np.random.choice(self.action_space)

        return action

    def decrement_epsilon(self):
        self.epsilon = self.epsilon * self.eps_dec \
                           if self.epsilon > self.eps_min else self.eps_min

    def learn(self, state, action, reward, state_):
        self.Q.optimizer.zero_grad()
        states = T.tensor(state, dtype=T.float).to(self.Q.device)
        actions = T.tensor(action).to(self.Q.device)
        rewards = T.tensor(reward).to(self.Q.device)
        states_ = T.tensor(state_, dtype=T.float).to(self.Q.device)

        q_pred = self.Q.forward(states)[actions]

        q_next = self.Q.forward(states_).max()

        q_target = rewards + self.gamma * q_next

        loss = self.Q.loss(q_target, q_pred).to(self.Q.device)
        loss.backward()
        self.Q.optimizer.step()
        self.decrement_epsilon()



#Testing the Agent in the CartPole Environment
env = gym.make('Acrobot-v1')
n_states = env.observation_space.shape
n_actions = env.action_space.n


gamma = 0.99
epsilon = 1.0
eps_dec = 0.9995
eps_min = 0.01

hidden = 128
lr = 0.0001

n_games = 1000


```

```
scores = []
eps_history = []

#n_states, n_actions, lr, alpha , gamma, epsilon, eps_dec, eps_min
#alpha, lr, n_states, n_actions

dqnAgent = DuelingModelAgent()

for i in range(n_games):
    score = 0
    done = False
    obs = env.reset()

    while not done:
        action = dqnAgent.epsilon_greedy_action_selection(obs)
        obs_, reward, done, info = env.step(action)
        score += reward
        dqnAgent.learn(obs, action, reward, obs_)
        obs = obs_
    scores.append(score)
    eps_history.append(dqnAgent.epsilon)

    if i % 100 == 0:
        avg_score = np.mean(scores[-100:])
        #print('episode ', i, 'score %.1f avg score %.1f epsilon %.2f' % (score,
        ↪  avg_score, agent.epsilon))
```

```
#Calculating the running mean for a smooth visualization of the rewards
#Here we are calculating the every 20 episode for smoothing
window_size = 20
i = 0
# Initialize an empty list to store moving averages
moving_averages = [-500]*window_size

while i < len(scores) - window_size + 1:

    # Calculate the average of current window
    window_average = round(np.sum(scores[i:i+window_size]) / window_size, 2)

    # Store the average of current
    # window in moving average list
    moving_averages.append(window_average)

    # Shift window to right by one position
    i += 1


#Plots to see the decay in the epsilon and rewards that agent collected
fig,ax1 = plt.subplots(figsize = (15,5))
plt.title('Epsilon Decay and Rewards through time')

color = 'tab:red'
ax1.set_xlabel('Episodes')
ax1.set_ylabel('Rewards', color=color)
ax1.plot(scores, color=color)
ax1.plot(moving_averages, color='magenta')
ax1.tick_params(axis='y', labelcolor=color)

ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis
color = 'tab:blue'
```

```
ax2.set_ylabel('Epsilon Decay', color=color)  # we already handled the x-label with ax1
ax2.plot(eps_history, color=color)
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()  # otherwise the right y-label is slightly clipped
plt.show()
```

```
class DuelingExpRepModel(nn.Module):
    def __init__(self,lr,n_actions,n_states):

        super(DuelingExpRepModel, self).__init__()
        #Initializing the dueling structure
        #State-Dependent action advantages stream
        self.adv1 = nn.Linear(*n_states, hidden)
        self.adv2 = nn.Linear(hidden, n_actions)
        #State-Value stream
        self.val1 = nn.Linear(*n_states, hidden)
        self.val2 = nn.Linear(hidden, 1)

        self.optimizer = optim.Adam(self.parameters(),lr=lr)
        self.loss = nn.MSELoss()
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')
        self.to(self.device)

    def forward(self,state):
        #Forward propagation with dueling architecture
        adv = nn.functional.relu(self.adv1(state))
        adv = self.adv2(adv)
        val = nn.functional.relu(self.val1(state))
        val = self.val2(val)

        actions = val + adv - adv.mean()
        #The Advantage is a quantity that obtained by subtracting the Q-value, by the
        ↪  V-value:
        return  actions


class DuelingExpRepModelAgent():
    def __init__(self):

        self.lr = lr
        self.n_states = n_states
        self.n_actions = n_actions
        self.gamma = gamma
        self.epsilon = epsilon
        self.eps_dec = eps_dec
        self.eps_min = eps_min
        self.action_space = [i for i in range(self.n_actions)]

        self.Q = DuelingExpRepModel(self.lr, self.n_actions, self.n_states)

    def epsilon_greedy_action_selection(self, observation):
        if np.random.random() > self.epsilon:
            state = T.tensor(observation, dtype=T.float).to(self.Q.device)
            actions = self.Q.forward(state)
            action = T.argmax(actions).item()
        else:
            action = np.random.choice(self.action_space)

        return action
```

```
53
54      def epsilon_update(self):
55          self.epsilon = self.epsilon * self.eps_dec \
56                              if self.epsilon > self.eps_min else self.eps_min
57
58
59      def learn_from_exp(self, state, action, reward, state_, memory, replay_size, gamma):
60          if len(memory) >= replay_size:
61
62              experience = random.sample(memory, replay_size)
63
64
65              predictions = []
66              targets = []
67              for state, action, reward, state_, done in experience:
68
69                  states = T.tensor(state, dtype=T.float).to(self.Q.device)
70                  actions = T.tensor(action).to(self.Q.device)
71                  rewards = T.tensor(reward).to(self.Q.device)
72                  states_ = T.tensor(state_, dtype=T.float).to(self.Q.device)
73                  done = T.tensor(state_, dtype=T.bool).to(self.Q.device)
74
75                  predictions.append(state)
76                  q_pred = self.Q.forward(predictions).tolist()
77
78                  if done:
79                      #q_target = q_pred
80                      q_target = q_pred.max()
81                  else:
82                      q_next = self.Q.forward(states_).max()
83                      q_target =reward + self.gamma * q_next
84
85                  targets.append(q_target)
86                  q_target = self.Q.forward(targets).tolist()
87
88
89              self.Q.optimizer.zero_grad()
90
91              loss = self.Q.loss(targets, states).to(self.Q.device)
92              loss.backward()
93              self.Q.optimizer.step()
94          self.epsilon_update()
95
96
97  #Testing the Agent in the CartPole Environment
98  env = gym.make('Acrobot-v1')
99  n_states = env.observation_space.shape
100 n_actions = env.action_space.n
101
102
103 gamma = 0.9
104 epsilon = 1.0
105 eps_dec = 0.9977
106 eps_min = 0.01
107
108 hidden = 128
109 lr = 0.001
110
111 n_games = 1000
112
113
```

```
scores = []
eps_history = []


agent = DuelingExpRepModelAgent()
memory = deque(maxlen=100)
replay_size = 20



for i in range(n_games):
    score = 0
    done = False
    obs = env.reset()

    while not done:
        action = agent.epsilon_greedy_action_selection(obs)
        obs_, reward, done, info = env.step(action)
        score += reward
        agent.learn_from_exp(obs, action, reward, obs_,memory, replay_size, gamma)
        obs = obs_
    scores.append(score)
    eps_history.append(agent.epsilon)

    if i % 100 == 0:
        avg_score = np.mean(scores[-100:])
        #print('episode ', i, 'score %.1f avg score %.1f epsilon %.2f' % (score,
        ↪  avg_score, agent.epsilon))
```

```
#Calculating the running mean for a smooth visualization of the rewards
#Here we are calculating the every 20 episode for smoothing
window_size = 20
i = 0
# Initialize an empty list to store moving averages
moving_averages = [-500]*window_size

while i < len(scores) - window_size + 1:

    # Calculate the average of current window
    window_average = round(np.sum(scores[i:i+window_size]) / window_size, 2)

    # Store the average of current
    # window in moving average list
    moving_averages.append(window_average)

    # Shift window to right by one position
    i += 1


#Plots to see the decay in the epsilon and rewards that agent collected
fig,ax1 = plt.subplots(figsize = (15,5))
plt.title('Epsilon Decay and Rewards through time')

color = 'tab:red'
ax1.set_xlabel('Episodes')
ax1.set_ylabel('Rewards', color=color)
ax1.plot(scores, color=color)
ax1.plot(moving_averages, color='magenta')
ax1.tick_params(axis='y', labelcolor=color)

```

```
ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis
color = 'tab:blue'
ax2.set_ylabel('Epsilon Decay', color=color)  # we already handled the x-label with ax1
ax2.plot(eps_history, color=color)
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()  # otherwise the right y-label is slightly clipped
plt.show()
```

```
# Task 9 Aziz
# Some Config parameters taken from
# https://docs.ray.io/en/latest/rllib/rllib-algorithms.html
# And the layout of the file is from INM707 Lab 9

!pip install gputil
!pip install dm_tree
!pip install lz4
!pip install ray[rllib]
!pip install gym[atari,accept-rom-license]

import ray
import ray.rllib.agents.dqn as dqn
from ray import tune
import matplotlib.pyplot as plt


#ray.init(num_cpus=3, ignore_reinit_error=True, log_to_driver=False)
config = dqn.DEFAULT_CONFIG.copy()
config["env"] = 'Pong-ram-v4'

config["framework"] = "torch"
config["num_atoms"] = 1
config["v_min"] = -10.0
config["v_max"] = 10.0
config["dueling"] = False
config["double_q"] = True
config["noisy"] = False
config["sigma0"] = 0.1
config["n_step"] = 10
#config["simple_optimizer"] = True
#config["disable_env_checking"]  = True
config["gamma"] = 0.99
config["lr"] = 0.001
# === Prioritized replay buffer ===
config["prioritized_replay"] = True
config["prioritized_replay_alpha"] = 0.6
config["prioritized_replay_beta"] = 0.4
config["final_prioritized_replay_beta"] = 0.4
config["prioritized_replay_beta_annealing_timesteps"] = 20000
config["prioritized_replay_eps"] = 1e-6
config["final_prioritized_replay_beta"] = 0.4
config["model"] = { "fcnet_hiddens": [256],
                    "fcnet_activation": "relu",
    }


trainer = dqn.DQNTrainer(config=config) # For fresh start
#trainer.restore("/root/ray_results/DQNTrainer_Pong-ram-v4_2022-04-18_14-14-08xyt53iej/checkpoint_000091/ch

#trainer.restore("/root/ray_results/DQNTrainer_Pong-ram-v4_2022-04-18_14-43-596liee4yd/checkpoint_000991/ch
```

```
avg_rewards = []
episodes = 3500
for episode in range(episodes):
    # Perform one iteration of training the policy with PPO
    result = trainer.train()
    print("Episode ", episode, ": ", "Score: ", result['episode_reward_mean'])
    avg_rewards.append(result['episode_reward_mean'])

    if episode % 10 == 0 and episode > 1:
        checkpoint = trainer.save()
        print("checkpoint saved at", checkpoint)


avg_rewards_1000EPS = avg_rewards

plt.title("Score per Episode for Pong Ram ")
plt.xlabel("Episodes")
plt.ylabel("Return (Score)")
plt.plot(avg_rewards, 'b')


# DO NOT TOUCH # plt.plot(avg_rewards, 'b') # OLD PLOT


config["env"] = 'Pong-ram-v4'
config["framework"] = "torch"

#config["dueling"] = tune.grid_search([True,False])
config["dueling"] = False

#config["double_q"] = tune.grid_search([True,False])
config["double_q"] = True

#config["noisy"] = tune.grid_search([True,False])
#config["prioritized_replay"] = tune.grid_search([True,False])

#config["gamma"] = tune.grid_search([0.9, 0.99])
config["gamma"] = 0.9

#config["lr"] = tune.grid_search([0.1, 0.01, 0.001, 0.0001])
config["lr"] = 0.001

analysis = tune.run('DQN',metric="episode_reward_mean", num_samples=1,config=config)

# Run these one by one for quicker results

config["env"] = 'Pong-ram-v4'
#config["framework"] = "torch"
#config["dueling"] = tune.grid_search([True,False])
#config["double_q"] = tune.grid_search([True,False])
#config["noisy"] = tune.grid_search([True,False])
#config["prioritized_replay"] = tune.grid_search([True,False])
config["gamma"] = tune.grid_search([0.9, 0.99])
#config["lr"] = tune.grid_search([0.01, 0.001, 0.0001])




# print(config)

analysis = tune.run('DQN',metric="episode_reward_mean", num_samples=1,config=config)
```

Extra Task

```python
# Code mostly from http://www.sagargv.com/blog/pong-ppo/

class Policy(nn.Module):
    def __init__(self):
        super(Policy, self).__init__()

        self.gamma = 0.99
        self.eps_clip = 0.1

        self.layers = nn.Sequential(
            nn.Linear(6000, 512), nn.ReLU(),
            nn.Linear(512, 2),
        )

    def state_to_tensor(self, I):
        """ prepro 210x160x3 uint8 frame into 6000 (75x80) 1D float vector. See Karpathy's post: http://karpathy.github.io/2016/05/31/rl/ """
        if I is None:
            return torch.zeros(1, 6000)
        I = I[35:185] # crop - remove 35px from start & 25px from end of image in x, to reduce redundant parts of image (i.e. after ball passes paddle)
        I = I[::2,::2,0] # downsample by factor of 2.
        I[I == 144] = 0 # erase background (background type 1)
        I[I == 109] = 0 # erase background (background type 2)
        I[I != 0] = 1 # everything else (paddles, ball) just set to 1. this makes the image grayscale effectively
        return torch.from_numpy(I.astype(np.float32).ravel()).unsqueeze(0)

    def pre_process(self, x, prev_x):
        return self.state_to_tensor(x) - self.state_to_tensor(prev_x)

    def convert_action(self, action):
        return action + 2

    def forward(self, d_obs, action=None, action_prob=None, advantage=None, deterministic=False):
        if action is None:
            with torch.no_grad():
                logits = self.layers(d_obs)
                if deterministic:
                    action = int(torch.argmax(logits[0]).detach().cpu().numpy())
                    action_prob = 1.0
                else:
                    c = torch.distributions.Categorical(logits=logits)
                    action = int(c.sample().cpu().numpy()[0])
                    action_prob = float(c.probs[0, action].detach().cpu().numpy())
                return action, action_prob
        '''
        # policy gradient (REINFORCE)
        logits = self.layers(d_obs)
        loss = F.cross_entropy(logits, action, reduction='none') * advantage
        return loss.mean()
        '''

        # PPO
        vs = np.array([[1., 0.], [0., 1.]])
        ts = torch.FloatTensor(vs[action.cpu().numpy()])

        logits = self.layers(d_obs)
```

```
        r = torch.sum(F.softmax(logits, dim=1) * ts, dim=1) / action_prob
        loss1 = r * advantage
        loss2 = torch.clamp(r, 1-self.eps_clip, 1+self.eps_clip) * advantage
        loss = -torch.min(loss1, loss2)
        loss = torch.mean(loss)

        return loss
```

```
env = gym.make('PongNoFrameskip-v4')
env.reset()

policy = Policy()

opt = torch.optim.Adam(policy.parameters(), lr=1e-3)
all_reward = []
run_ave = []
reward_sum_running_avg = None
for it in range(50):
    d_obs_history, action_history, action_prob_history, reward_history = [], [], [], []
    for ep in range(10):
        obs, prev_obs = env.reset(), None
        for t in range(190000):
            #env.render()

            d_obs = policy.pre_process(obs, prev_obs)
            with torch.no_grad():
                action, action_prob = policy(d_obs)

            prev_obs = obs
            obs, reward, done, info = env.step(policy.convert_action(action))

            d_obs_history.append(d_obs)
            action_history.append(action)
            action_prob_history.append(action_prob)
            reward_history.append(reward)

            if done:
                reward_sum = sum(reward_history[-t:])
                reward_sum_running_avg = 0.99*reward_sum_running_avg + 0.01*reward_sum
                ↪  if reward_sum_running_avg else reward_sum
                all_reward.append(reward_sum)
                run_ave.append(reward_sum_running_avg)
                print('Iteration %d, Episode %d (%d timesteps) - last_action: %d,
                ↪  last_action_prob: %.2f, reward_sum: %.2f, running_avg: %.2f' % (it,
                ↪  ep, t, action, action_prob, reward_sum, reward_sum_running_avg))
                break

    # compute advantage
    R = 0
    discounted_rewards = []

    for r in reward_history[::-1]:
        if r != 0: R = 0 # scored/lost a point in pong, so reset reward sum
        R = r + policy.gamma * R
        discounted_rewards.insert(0, R)

    discounted_rewards = torch.FloatTensor(discounted_rewards)
    discounted_rewards = (discounted_rewards - discounted_rewards.mean()) /
    ↪  discounted_rewards.std()

```

```
    # update policy
    for _ in range(5):
        n_batch = 24576
        idxs = random.sample(range(len(action_history)), n_batch)
        d_obs_batch = torch.cat([d_obs_history[idx] for idx in idxs], 0)
        action_batch = torch.LongTensor([action_history[idx] for idx in idxs])
        action_prob_batch = torch.FloatTensor([action_prob_history[idx] for idx in
        ↪  idxs])
        advantage_batch = torch.FloatTensor([discounted_rewards[idx] for idx in idxs])

        opt.zero_grad()
        loss = policy(d_obs_batch, action_batch, action_prob_batch, advantage_batch)
        loss.backward()
        opt.step()

    if it % 5 == 0:
        torch.save(policy.state_dict(), 'params.ckpt')

env.close()
```

```
import matplotlib.pyplot as plt

plt.plot(all_reward, c='red')
plt.plot(run_ave, c='blue')
plt.title('Rewards')
plt.xlabel('Episodes')
plt.ylabel('Rewards through Episodes')
plt.show()

plt.plot(discounted_rewards, c='orange')
plt.title('Discounted Reward Q')
plt.xlabel('Timesteps')
plt.ylabel('Discounted Reward Q through timesteps')
plt.show()
```